

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer, fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier
```

```
In [8]: data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

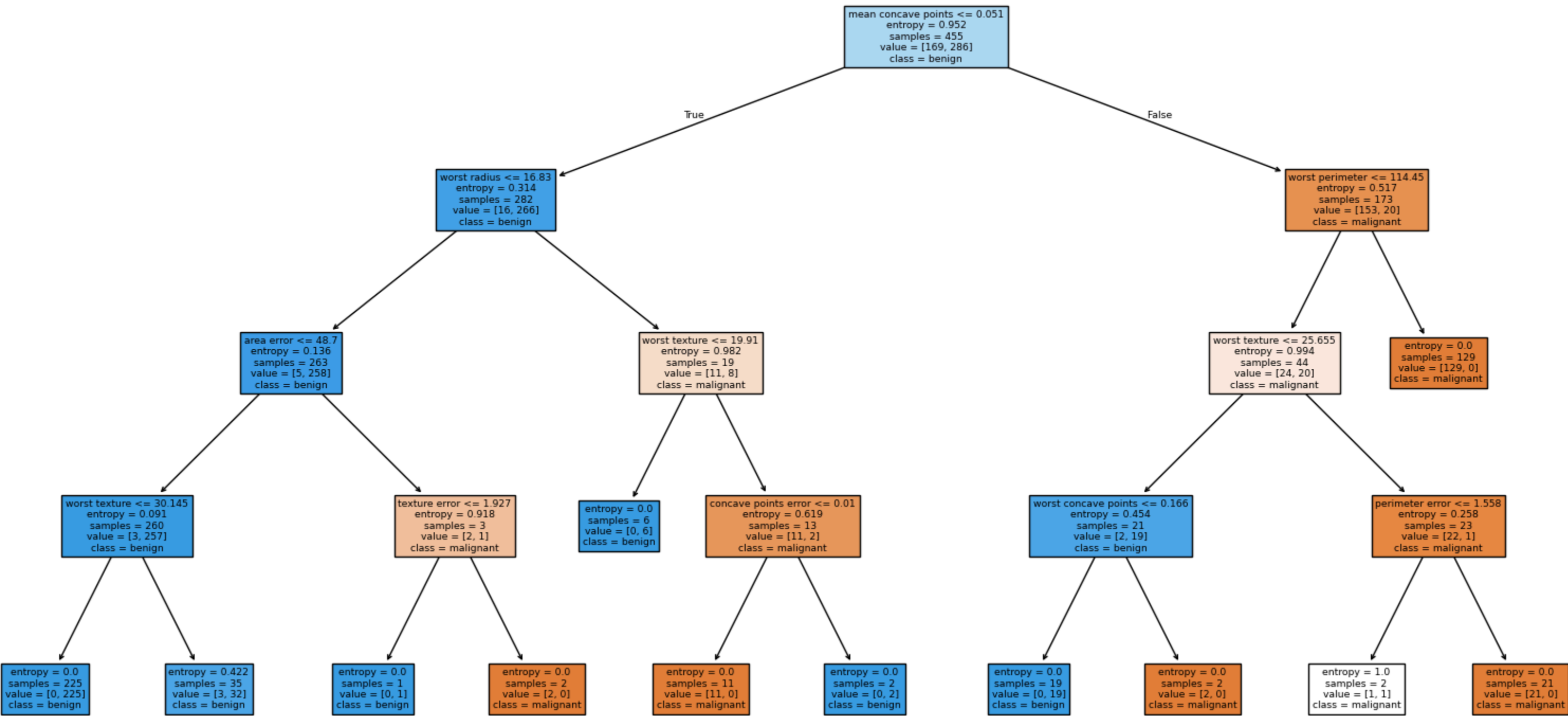
dt_clf = DecisionTreeClassifier(criterion='entropy', max_depth=4, random_state=42)
dt_clf.fit(X_train, y_train)

print("--- Класифікація (Decision Tree) ---")
print(f"Train Accuracy: {dt_clf.score(X_train, y_train):.3f}")
print(f"Test Accuracy: {dt_clf.score(X_test, y_test):.3f}")

plt.figure(figsize=(20, 10))
plot_tree(dt_clf, feature_names=data.feature_names, class_names=data.target_names, filled=True)
plt.title("Decision Tree Structure")
plt.show()
```

--- Класифікація (Decision Tree) ---
Train Accuracy: 0.991
Test Accuracy: 0.956

Decision Tree Structure



```
In [9]: print("--- Ансамблеві методи ---")

rf_clf = RandomForestClassifier(n_estimators=100, max_depth=4, random_state=42)
rf_clf.fit(X_train, y_train)
print(f"Random Forest Accuracy: {rf_clf.score(X_test, y_test):.3f}")

gb_clf = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
gb_clf.fit(X_train, y_train)
print(f"Gradient Boosting Accuracy: {gb_clf.score(X_test, y_test):.3f}")
```

--- Ансамблеві методи ---
Random Forest Accuracy: 0.965
Gradient Boosting Accuracy: 0.956

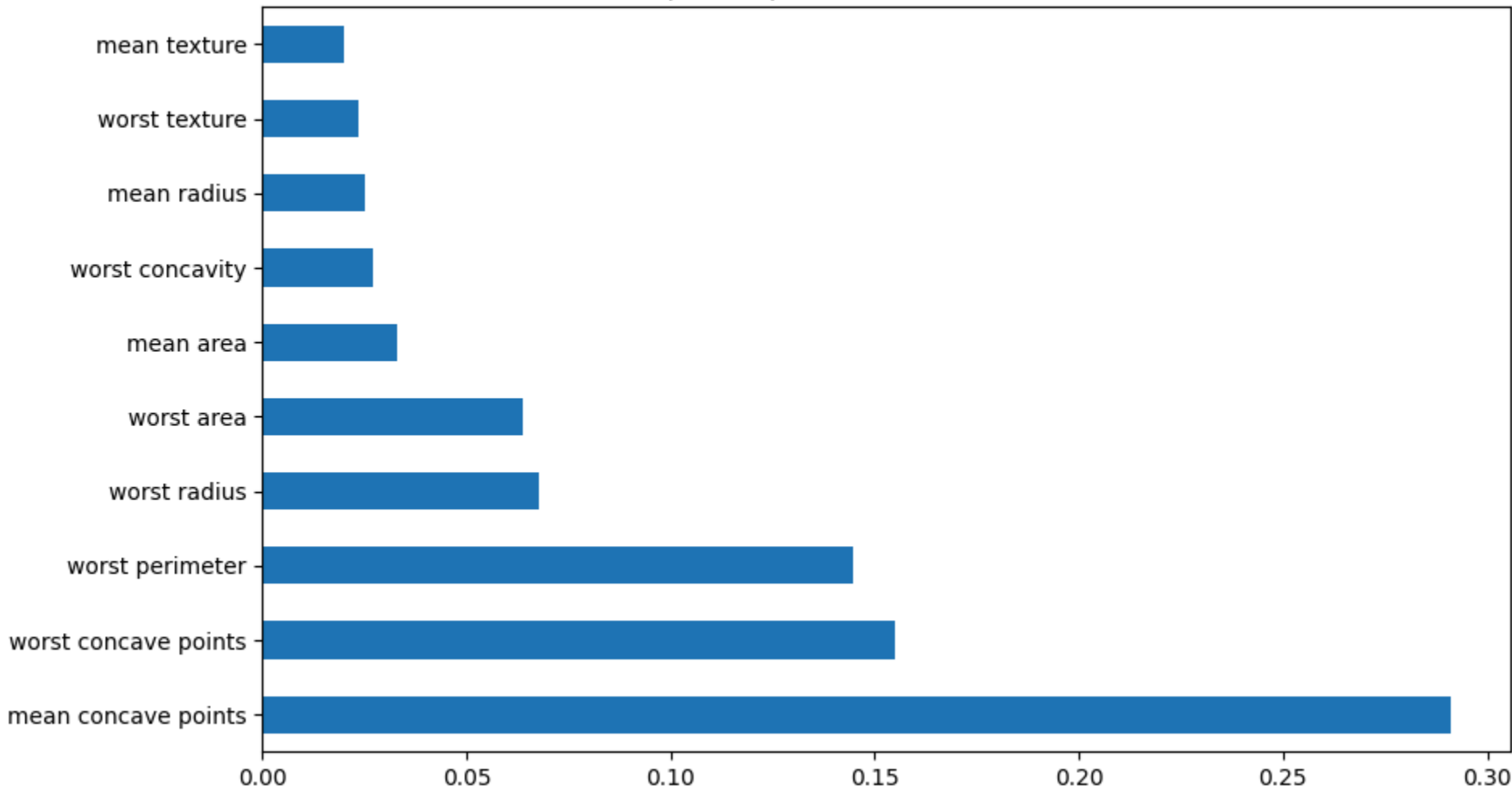
```
In [5]: xgb_clf = XGBClassifier(n_estimators=100, learning_rate=0.05, verbosity=0, use_label_encoder=False)
xgb_clf.fit(X_train, y_train)

xgb_pred = xgb_clf.predict(X_test)
print(f"XGBoost Accuracy: {accuracy_score(y_test, xgb_pred):.3f}")

plt.figure(figsize=(10, 6))
feat_importances = pd.Series(xgb_clf.feature_importances_, index=data.feature_names)
feat_importances.nlargest(10).plot(kind='barh')
plt.title("Top 10 Important Features (XGBoost)")
plt.show()
```

XGBoost Accuracy: 0.956

Top 10 Important Features (XGBoost)



```
In [10]: housing = fetch_openml(name="house_prices", as_frame=True, parser='auto')
X_reg = housing.data.select_dtypes(include=[np.number]).fillna(0)
y_reg = housing.target

X_r_train, X_r_test, y_r_train, y_r_test = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
```

```
dt_reg = DecisionTreeRegressor(max_depth=5, random_state=42)
dt_reg.fit(X_r_train, y_r_train)

print("--- Перспекія (House Prices) ---")
print(f"R^2 Score on Test: {dt_reg.score(X_r_test, y_r_test):.3f}")

--- Перспекія (House Prices) ---
R^2 Score on Test: 0.795
```